

Universidad Galileo

Ciencias de la Computación VIII

Ing. Axel Benavides

## **Proyecto No. 1 – Captura la Bandera**

### **Documentación de Implementación**

Adrian Esteban Romero Silvestre

23004161

Sección A

## Historial de Versiones y Evolución del Proyecto

Dentro de esta sección se detalla el avance de forma cronológica del desarrollo del proyecto documentando aspectos técnicos, cambios de ideas frente a los problemas de red e implementaciones.

### Fase 1: Arquitectura Base y Comunicación Inicial

**Fecha:** 19/07

#### Commits de referencia:

- 17aac15 - feat(network): definición de constantes, construcción de mensajes del protocolo e implementación del descubrimiento UDP cliente-servidor.
- 54df83b - feat (network): implementación cliente-servidor TCP y regla del buffer.

**Descripción y Evolución:** Al inicio del proyecto busco abordarse todo lo relacionado con establecer las bases de comunicación. Inicialmente, se implemento un servicio de descubrimiento local median Broadcast UDP y una conexión persistente TCP. Durante esta fase se definió la primera regla del buffer para evitar fraccionamiento de los datos TCP.

### Fase 2: Lógica Central (Game Engine) y Mecánicas

**Fecha:** 20/07 – 22/07

#### Commits de referencia:

- 18b20e6 - feat(server): implementación handshake join/welcome y generación de ID's únicos.
- d4abebe - feat(engine): Implementación del GameEngine y broadcasting del estado actual del jugador y bandera.
- 5692657 - feat(game\_logic): Implementación de mecánicas de interacción con la bandera, posiciones de spawn aleatorias y restricciones del mapa

**Descripción y Evolución:** Se implemento el motor lógico del servidor. La idea inicial de la mecánica de movimiento evoluciono incorporando sincronización basada en el tiempo real y restricciones de bordes del mapa. También se definió la interacción basada en un radio constante para capturar o robar la bandera, evitando tiempos de espera según lo que se estipulo en las reglas, además de un funcionamiento consistente.

### **Fase 3: Interfaz Grafica (GUI) y Maquina de Estados**

**Fecha:** 25/07, 29/07

#### **Commits de referencia:**

- 24ca7ba - feat(engine): Implementación máquina de estados para el flujo de la partida.
- d4bbeb4 - feat(client): Corresponde a los cambios realizados por parte del servidor para la interpretación de estados y funcionamiento dentro de la partida.
- ddae996 - feat(game): Interfaz gráfica tanto para cliente como para el servidor.
- 11866b9 - fix(game): Espectar a la hora de actuar como servidor.
- 629ad58 - fix(game\_logic): Volver al lobby despues de GAME\_OVER

**Descripción y Evolución:** El juego paso de ser funcional mediante la terminal a integrar una interfaz grafica hecha con la librería de Pygame. Se desarrollo una maquina de estados finitos o flujo normal dentro de la partida (LOBBY, COUNTDOWN, PLAYING, GAME\_OVER). El diseño evoluciono separando el renderizado del cliente (dibuja el mapa basado en el JSON) y renderizado del servidor (toma datos directamente de la memoria local).

### **Fase 4: Estandarización del Protocolo**

**Fecha:** 25/07

#### **Commits de referencia:**

- 3acff0c - fix(general): Correcciones varias relacionadas con el protocolo.
- 7daf99f - fix(protocol): Ajustes para apegarse al protocolo y correcto funcionamiento del juego cliente/servidor.

**Descripción y Evolución:** Fase crítica para garantizar la comunicación con otros proyectos.

Se reajustó la transmisión de datos para que el motor solo envíe actualizaciones de mapa (state) cuando el juego está en curso, evitando crashear clientes con validaciones estrictas. Además, se adaptó el cliente para cruzar el ID del mensaje game\_over con la lista local de jugadores, logrando mostrar los nombres dinámicamente en partida.

## Arquitectura de Red y Comunicación Multi-Proyecto

Para garantizar un funcionamiento total con los proyectos del “grupo”, se implemento una arquitectura cliente-servidor basada en un esquema de transporte hibrido. Se utiliza el protocolo UDP exclusivamente para la fase de descubrimiento dentro de la red local, y el protocolo TCP para asegurar la entrega completa y en orden de todos los eventos críticos de la partida. Para más información acerca del protocolo, se encuentra dentro de la misma carpeta carpeta de /docs en el repositorio. Aquí se explica por encima las conexiones.

### Fase 1: Descubrimiento de Servidores (UDP)

El juego incluye un "radar" de red local que permite a los clientes encontrar partidas sin conocer las direcciones IP previamente, utilizando el puerto fijo de descubrimiento 8888.

- **Broadcast del Cliente:** A través de la clase `DiscoveryScanner`, el cliente envía un mensaje empaquetado como `{"type": "discover", "v": 1}` hacia la dirección de difusión general 255.255.255.255.
- **Respuesta del Servidor:** La clase `DiscoveryService` del servidor escucha constantemente en el puerto 8888. Al recibir un `discover`, examina la IP de origen y responde directamente a ese remitente con un mensaje `server_info`, el cual contiene su nombre, estado y, fundamentalmente, el puerto TCP dinámico donde alojará la partida.
- **Conexión Manual (Respaldo):** Para entornos de red restrictivos o VPNs donde el tráfico broadcast es bloqueado, el escáner incluye un mecanismo de conexión manual que permite inyectar directamente la IP objetivo, cumpliendo con la regla de respaldo exigida por el estándar.

### Fase 2: Protocolo de Conexión y Gestión de Hilos (TCP)

Una vez descubierto el puerto TCP del servidor, el cliente abandona el uso de UDP y abre una conexión TCP persistente.

- **Gestión Multihilo:** Para cumplir el requisito de soportar hasta 100 jugadores simultáneos, la clase `TCPServer` genera un subproceso (`threading.Thread`)

exclusivo e independiente por cada cliente conectado. Esto asegura que la lectura de datos de un jugador no bloquee ni retrase al resto del servidor.

- **Apretón de Manos (Handshake):** El cliente inicia la comunicación enviando el mensaje `join` con su nombre en formato UTF-8. El servidor valida la versión, genera un identificador único (UUID) y responde con el mensaje `welcome`. Este mensaje de bienvenida transmite la identidad del jugador y las constantes de la partida, tales como el tamaño del mapa (1000) y las velocidades.

### **Fase 3: Delimitación de Mensajes y Regla del Búfer (Framing)**

Debido a la naturaleza de TCP como un flujo continuo de bytes, múltiples mensajes pueden llegar agrupados en un solo paquete o fragmentados. Para garantizar una lectura impecable, se implementó una estricta regla de estructuración plana.

- **Codificación:** Todo mensaje se serializa usando la librería `json` de Python, eliminando saltos de línea internos mediante la instrucción `separators=(',', ':')` y forzando la codificación a UTF-8.
- **Regla del Salto de Línea (`\n`):** En el proceso de codificación TCP (`encode_tcp_message`), se anexa el carácter `\n` al final de cada JSON. El servidor y el cliente almacenan los bytes entrantes en un búfer temporal de texto.
- **Procesamiento:** El sistema solo extrae y procesa (parsea) un mensaje cuando detecta el `\n` dentro del búfer, conservando el resto de los datos fragmentados para la siguiente lectura iterativa. Esto erradica la pérdida de paquetes o caídas del sistema por "JSON inválidos" cuando hay alta latencia.

## Registro de Asistencia de Inteligencia Artificial

Durante el ciclo de desarrollo, se utilizó asistencia de Inteligencia Artificial, en este caso, Gemini. Empleando técnicas de Prompt Engineering para guiar la creación de la arquitectura, depurar la lógica matemática y establecer los flujos de red.

A continuación, se documentan las interacciones clave, mostrando la evolución desde la definición de la arquitectura hasta la implementación gráfica:

### Fase 1: Arquitectura y Modulo Core

- Diseño Base: "Diseña la arquitectura interna del sistema cliente-servidor antes de generar el código. El enfoque principal debe ser mantener una estructura altamente modular, legible y escalable, separando claramente las responsabilidades de red y las del motor de juego."
- Implementación del Core: "Procede con la implementación del módulo 'Core' propuesto en la arquitectura. Genera el código correspondiente y acompáñalo de una explicación técnica detallada que describa el propósito y la función de cada componente, constante y protocolo de empaquetado de datos."

### Fase 2: Protocolos de Red (UDP y TCP)

- Descubrimiento UDP: "Continúa con el desarrollo del flujo de red. Tomando como base la implementación previa de `discovery_service.py` (Fase UDP), desarrolla el siguiente componente necesario para gestionar el descubrimiento de servidores locales."
- Transición a TCP: "Habiendo validado exitosamente el descubrimiento UDP, inicia la implementación de la Fase 2: la conexión persistente. Genera el código para `tcp_server.py`, asegurando que maneje conexiones entrantes y prepare la transición al protocolo TCP."
- Handshake: "Implementa el mensaje de apretón de manos ('handshake') en el servidor TCP. Refactoriza el código para que procese el mensaje `join` del cliente y responda con el mensaje estandarizado `welcome`, validando la conexión inicial."

### **Fase 3: Motor del Servidor y Físicas**

- Bucle Principal: "Diseña la estructura de la clase GameEngine para el servidor. Explica e implementa cómo se debe gestionar el bucle principal del juego para que opere a una frecuencia fija de 20Hz (ticks), garantizando una actualización de estado consistente para todos los clientes."
- Lógica de Movimiento: "Implementa el sistema de movimiento del jugador. Describe y codifica el flujo completo: la captura de los eventos WASD en el cliente, la serialización del mensaje input, y la actualización de las coordenadas en el servidor."
- Corrección Matemática (Vectores): "Revisa la lógica de movimiento vectorial en el servidor. ¿Cómo se está manejando la normalización de los vectores en los ejes X e Y al moverse en diagonal para evitar ventajas de velocidad? Implementa la corrección matemática si es necesario."

### **Fase 4: Reglas del Juego y Estados**

- Condición de Victoria: "Procede con la implementación del sistema de interacciones. Enfócate específicamente en definir y codificar la condición matemática de victoria (Game Over) calculada desde la autoridad del servidor."
- Reinicio de Partida: "Desarrolla el flujo de finalización de partida en game\_engine.py. Integra la detección de la condición de victoria, el envío del evento game\_over en broadcast a los clientes y la rutina para reiniciar automáticamente el estado lógico de la partida."

### **Fase 5: Interfaz Grafica y Experiencia de Usuario**

- Punto de Entrada Unificado: "Desarrolla un punto de entrada unificado (main.py) con una interfaz gráfica en Pygame. Este menú principal debe permitir al usuario seleccionar entre operar como 'Servidor' o 'Cliente', delegando la ejecución a server\_main o client\_main respectivamente, integrando el flujo completo de descubrimiento y conexión."
- Captura de Datos del Usuario: "Añade un componente de entrada de texto (Input Box) en la interfaz gráfica de Pygame del cliente. Este campo debe capturar el nombre del jugador ingresado para incluirlo en la carga útil (payload) del mensaje join al momento de conectar con el servidor TCP."

- Modo Espectador y Control de Flujo: "Implementa una vista de espectador pasiva en tiempo real y un control de estado para el host del servidor. Añade un botón de 'Iniciar Partida' en la interfaz del servidor para transicionar del estado 'lobby' a 'game'. Evalúa y describe previamente los cambios arquitectónicos necesarios para soportar este flujo de estados sincronizado."